

Notification System Flow

Overview

The Luciteria Notification System delivers personalized, context-aware alerts to collectors about collection milestones, wishlist restocks, and relevant product arrivals. This document details the trigger logic, filtering rules, message composition, and delivery mechanisms for all 5 notification types.

System Architecture

Components



Notification Types

1. Completion Unlock

Purpose: Alert collector when purchasing a specific element would unlock a milestone

Trigger Conditions:

- User is at X% completion (e.g., 89%)
- Purchasing 1 specific element would push them to next milestone (e.g., 90%)
- That element is currently in stock
- Element is NOT already on their wishlist (avoid duplicate notifications)

Example Scenario:

- Sarah has 88 elements (74.6% completion)
- Next milestone: 75% (requires 89 elements, just 1 more)
- Selenium (Se) is on her wishlist and in stock
- **Trigger:** "Adding Selenium would unlock your 75% milestone!"

Trigger Logic (pseudocode):

```
function detectCompletionUnlock(userId) {
  const ownedCount = await getOwnedElementCount(userId);
  const completion = (ownedCount / 118) * 100;

  // Find next milestone threshold
  const milestones = [10, 25, 50, 75, 90, 100];
  const nextMilestone = milestones.find(m => m > completion);

  if (!nextMilestone) return; // Already at 100%

  const elementsNeededForNext = Math.ceil((nextMilestone / 100) * 118) - ownedCount;

  // Only trigger if 1 element away
  if (elementsNeededForNext === 1) {
    const wishlistElements = await getUserWishlist(userId);
    const inStockWishlistElements = wishlistElements.filter(e => e.inStock);

    if (inStockWishlistElements.length > 0) {
      // Pick highest priority wishlist item
      const targetElement = inStockWishlistElements[0];

      await queueNotification({
        userId,
        type: 'COMPLETION_UNLOCK',
        relatedElementId: targetElement.id,
        metadata: {
          currentCompletion: completion,
          nextMilestone,
          elementSymbol: targetElement.symbol
        }
      });
    }
  }
}
```

Message Template:

Subject: 🎯 One Element Away from {next_milestone}% Completion!

Hi {user_name},

You're so close! You're currently at {current_completion}% completion with {owned_count} elements.

Adding {element_name} ({symbol}) to your collection would unlock your {next_milestone}% milestone.
It's in stock right now!

[View {Element_Name}] [Browse Your Cabinet]

Keep building your collection,
The Luciteria Team

Frequency Limit: Max 1 per week (avoid spamming as they approach milestones)

2. Near-Completion Alert

Purpose: Celebrate progress when collector reaches high completion in a category

Trigger Conditions:

- User owns 90%+ of elements in a specific category (e.g., Transition Metals, Halogens)
- User has NOT yet completed 100% of that category
- User has NOT received this notification for this category before

Categories (from periodic table):

- Alkali Metals (6 elements)
- Alkaline Earth Metals (6 elements)
- Transition Metals (38 elements)
- Post-Transition Metals (11 elements)
- Metalloids (7 elements)
- Nonmetals (7 elements)
- Halogens (5 elements)
- Noble Gases (7 elements)
- Lanthanides (15 elements)
- Actinides (15 elements)

Example Scenario:

- Michael owns 6 out of 7 Halogens (85.7%)
- Missing: Astatine (At)
- **Trigger:** "You're 1 element away from completing Halogens!"

Trigger Logic:

```
function detectNearCompletion(userId) {
  const categories = await getElementCategories();

  for (const category of categories) {
    const ownedInCategory = await getOwnedElementsInCategory(userId, category.name);
    const totalInCategory = category.elementCount;
    const completion = (ownedInCategory.length / totalInCategory) * 100;

    if (completion >= 90 && completion < 100) {
      const alreadyNotified = await checkNotificationSent(userId, 'NEAR_COMPLETION', category.name);

      if (!alreadyNotified) {
        const missingElements = category.elements.filter(e => !ownedInCategory.includes(e));

        await queueNotification({
          userId,
          type: 'NEAR_COMPLETION',
          metadata: {
            categoryName: category.name,
            completion,
            ownedCount: ownedInCategory.length,
            totalCount: totalInCategory,
            missingElements: missingElements.map(e => e.symbol)
          }
        });
      }
    }
  }
}
```

Message Template:

Subject: 🏆 You're Almost There! {completion}% of {category_name}

Hi {user_name},

Impressive progress! You've collected {owned_count} out of {total_count} {category_name}.

You're just {missing_count} element(s) away from completing this category: {missing_elements_list}

[View {Category_Name}] [Browse Your Cabinet]

You're doing amazing!
The Luciteria Team

Frequency Limit: 1-time per category (never repeat once sent)

3. Wishlist Restock Alert

Purpose: Notify when a wishlisted item comes back in stock

Trigger Conditions:

- Product inventory changes from 0 → >0 (or availableForSale false → true)

- Product's element is on user's wishlist
- User has NOT been notified about this restock in last 30 days

Example Scenario:

- Jennifer has Gallium (Ga) on her wishlist
- Gallium 25.4mm was out of stock
- Supplier restocks, inventory updated to 15 units
- **Trigger:** "Good news! Gallium is back in stock."

Trigger Logic:

```

async function detectRestocks(productId) {
  // Called when product inventory updated
  const product = await getProduct(productId);

  if (product.inventory > 0 && product.availableForSale) {
    const elementSymbol = parseElementFromProduct(product);
    const usersWaitlisted = await getUsersWithWishlist(elementSymbol);

    for (const user of usersWaitlisted) {
      const recentlyNotified = await checkNotificationSent(
        user.id,
        'WISHLIST_RESTOCK',
        elementSymbol,
        { withinDays: 30 }
      );

      if (!recentlyNotified) {
        await queueNotification({
          userId: user.id,
          type: 'WISHLIST_RESTOCK',
          relatedProductId: product.id,
          metadata: {
            elementSymbol,
            elementName: product.elementName,
            format: product.format,
            price: product.price
          }
        });
      }
    }
  }
}

```

Message Template:

Subject: ✨ Great News! {Element_Name} is Back in Stock

Hi {user_name},

The {element_name} ({symbol}) {format} you wishlisted is now available!

We just restocked and wanted to let you know before it sells out again.

Price: \${price}
Format: {format}

[Add to Cart] [View Product Details]

Happy collecting,
The Luciteria Team

Frequency Limit: Max 1 per element per 30 days (avoid spam if item goes in/out of stock frequently)

4. High-Relevance Product Arrival

Purpose: Alert about new products highly relevant to their collection

Trigger Conditions:

- New product added to catalog
- Product's element is:
 - * On user's wishlist, OR
 - * Missing from their collection AND in a category they actively collect (50%+ owned), OR
 - * A rare/valuable element (<5% of collectors own it)
- Product format matches user's preferred format (from profile)
- User has NOT received a "new arrival" notification in last 7 days (general rate limit)

Relevance Scoring:

```
function calculateRelevanceScore(userId, product) {
  let score = 0;

  // On wishlist: +100 points (highest priority)
  if (isOnWishlist(userId, product.elementSymbol)) score += 100;

  // Missing from collection: +50 points
  if (!ownsElement(userId, product.elementSymbol)) score += 50;

  // In actively collected category: +30 points
  const categoryCompletion = getCategoryCompletion(userId, product.category);
  if (categoryCompletion >= 50) score += 30;

  // Rare element: +20 points
  const globalOwnership = getGlobalOwnershipPercent(product.elementSymbol);
  if (globalOwnership < 5) score += 20;

  // Format match: +10 points
  if (product.format === getUserPreferredFormat(userId)) score += 10;

  return score;
}
```

Example Scenario:

- New product added: Rhodium (Rh) 50mm cube
- David has Rh on wishlist, prefers 50mm format
- Relevance score: 100 (wishlist) + 10 (format) = 110
- **Trigger:** "New arrival: Rhodium 50mm cube—on your wishlist!"

Trigger Logic:

```

async function detectHighRelevanceArrivals(productId) {
  const product = await getProduct(productId);
  const allUsers = await getAllUsers();

  const relevantUsers = [];

  for (const user of allUsers) {
    const score = calculateRelevanceScore(user.id, product);

    if (score >= 80) { // Threshold for "high relevance"
      relevantUsers.push({ user, score });
    }
  }

  // Sort by score, notify highest relevance first
  relevantUsers.sort((a, b) => b.score - a.score);

  for (const { user, score } of relevantUsers) {
    const recentlyNotified = await checkNotificationSent(
      user.id,
      'HIGH_RELEVANCE_ARRIVAL',
      null,
      { withinDays: 7 }
    );

    if (!recentlyNotified) {
      await queueNotification({
        userId: user.id,
        type: 'HIGH_RELEVANCE_ARRIVAL',
        relatedProductId: product.id,
        metadata: {
          relevanceScore: score,
          elementSymbol: product.elementSymbol,
          format: product.format,
          price: product.price
        }
      });
    }
  }
}

```

Message Template (varies by score):

For Wishlist Match (score 100+):

Subject: 📦 New Arrival: {Element_Name} 📦 On Your Wishlist!

Hi {user_name},

Exciting news! We just added a new {element_name} ({symbol}) product to our catalog:

{Product_Name}
Format: {format}
Price: \${price}

This element is on your wishlist📦grab it before it's gone!

[View Product] [Your Wishlist]

Cheers,
 The Luciteria Team

For Category Match (score 80-99):

Subject: 📦 New {Category_Name} Element Available

Hi {user_name},

We noticed you're building a **strong** {category_name} collection ({completion}% complete).

We just added {element_name} ({symbol}) **in** {format} format:

Price: \${price}

[View Product] [Browse {Category_Name}]

Happy hunting,
 The Luciteria Team

Frequency Limit: Max 1 per week (avoid overwhelming with too many “new arrival” alerts)

5. Milestone Celebration

Purpose: Celebrate when collector reaches a major milestone

Trigger Conditions:

- User's owned element count crosses a milestone threshold (10%, 25%, 50%, 75%, 90%, 100%)
- Milestone has NOT been celebrated before (1-time per milestone)

Milestone Thresholds:

Milestone	Elements Required	Percentage
----- ----- -----		
Starter	12 elements	10%
Enthusiast	30 elements	25%
Dedicated	59 elements	50%
Advanced	89 elements	75%
Master	106 elements	90%
Complete	118 elements	100%

Example Scenario:

- Emma adds her 59th element
- Crosses 50% threshold
- **Trigger:** “🎉 Milestone Unlocked: 50% Collection Complete!”

Trigger Logic:

```

async function detectMilestoneCompletion(userId, newlyAddedElementId) {
  const ownedCount = await getOwnedElementCount(userId);
  const completion = (ownedCount / 118) * 100;

  const milestones = [
    { name: 'Starter', threshold: 10, elements: 12 },
    { name: 'Enthusiast', threshold: 25, elements: 30 },
    { name: 'Dedicated', threshold: 50, elements: 59 },
    { name: 'Advanced', threshold: 75, elements: 89 },
    { name: 'Master', threshold: 90, elements: 106 },
    { name: 'Complete', threshold: 100, elements: 118 }
  ];

  for (const milestone of milestones) {
    if (ownedCount === milestone.elements) {
      const alreadyCelebrated = await checkMilestoneCelebrated(userId, milestone.name);

      if (!alreadyCelebrated) {
        await queueNotification({
          userId,
          type: 'MILESTONE_Celebration',
          metadata: {
            milestoneName: milestone.name,
            threshold: milestone.threshold,
            ownedCount,
            newlyAddedElementId
          }
        });

        // Mark milestone as celebrated
        await markMilestoneCelebrated(userId, milestone.name);
      }
    }
  }
}

```

Message Template:**For 10-75% Milestones:**

Subject: 🎉 Milestone Unlocked: {milestone_name} Collector!

Hi {user_name},

Congratulations! You've just reached {threshold}% completion with {owned_count} elements.

You're officially a {milestone_name} Collector! 🏆

Your collection is growing beautifully. Keep exploring the periodic table!

[View Your Cabinet] [Set New Goals]

Celebrate this achievement!
The Luciteria Team

For 90% Milestone:

Subject: 🌟 Amazing! You're at 90% Completion!

Hi {user_name},

This is incredible! You've collected 106 **out** of 118 elements.

You're **in** the top tier of collectors. Only 12 elements left to complete your collection!

Missing Elements: {list_of_missing_12}

[View Missing Elements] [Your Cabinet]

You're almost there!
The Luciteria Team

For 100% Milestone:

Subject: 🏆 LEGENDARY: 100% Collection Complete!

Hi {user_name},

🎉 YOU DID IT! 🎉

You've collected ALL 118 elements of the periodic table. This is an extraordinary achievement.

You're now part of an elite group of collectors who have completed the entire set.

[View Your Complete Collection] [Share Your Achievement]

From all of us at Luciteria: CONGRATULATIONS!

The Luciteria Team

Frequency Limit: 1-time per milestone (never repeated)

Notification Queue & Delivery

Database Schema

NotificationQueue Table

```

model NotificationQueue {
  id                String    @id @default(cuid())
  userId            String
  type              NotificationType
  status            NotificationStatus @default(PENDING)
  relatedElementId  String?
  relatedProductId  String?
  metadata          Json?
  relevanceScore     Int?
  createdAt         DateTime @default(now())
  scheduledFor       DateTime?
  sentAt            DateTime?
  deliveryMethod     String[] // ['in_app', 'email']

  user              User      @relation(fields: [userId], references: [id])

  @@index([userId, status])
  @@index([scheduledFor])
}

enum NotificationType {
  COMPLETION_UNLOCK
  NEAR_COMPLETION
  WISHLIST_RESTOCK
  HIGH_RELEVANCE_ARRIVAL
  MILESTONE_CELEBRATION
}

enum NotificationStatus {
  PENDING
  SENT
  FAILED
  CANCELLED
}

```

UserNotificationPreferences Table

```

model UserNotificationPreferences {
  id String @id @default(cuid())
  userId String @unique

  // Email preferences (per notification type)
  emailCompletionUnlock Boolean @default(true)
  emailNearCompletion Boolean @default(true)
  emailWishlistRestock Boolean @default(true)
  emailHighRelevance Boolean @default(true)
  emailMilestone Boolean @default(true)

  // In-app preferences
  inAppCompletionUnlock Boolean @default(true)
  inAppNearCompletion Boolean @default(true)
  inAppWishlistRestock Boolean @default(true)
  inAppHighRelevance Boolean @default(true)
  inAppMilestone Boolean @default(true)

  // General settings
  emailFrequencyLimit Int @default(3) // Max emails per week
  mutedUntil DateTime?

  user User @relation(fields: [userId], references: [id])
}

```

Queue Processing Flow



Frequency Limiting Logic

Per-Type Limits (daily):

| Type | Max per Day | Max per Week |

|-----|-----|-----|

Completion Unlock	1	1
Near-Completion	1	1
Wishlist Restock	3	10
High-Relevance	1	1
Milestone	Unlimited	Unlimited

Global Limit:

- Max 3 emails per week (configurable in preferences)
- In-app notifications: no global limit (user can scroll/dismiss)

Implementation:

```
async function checkFrequencyLimit(userId, notificationType) {
  const preferences = await getUserNotificationPreferences(userId);

  // Check type-specific limit
  const typeLimit = TYPE_FREQUENCY_LIMITS[notificationType];
  const sentInPeriod = await countNotificationsSent(
    userId,
    notificationType,
    typeLimit.period
  );

  if (sentInPeriod >= typeLimit.max) {
    return { allowed: false, reason: 'Type frequency limit exceeded' };
  }

  // Check global email limit
  if (deliveryMethod === 'email') {
    const emailsSentThisWeek = await countNotificationsSent(
      userId,
      null, // all types
      '7 days',
      'email'
    );

    if (emailsSentThisWeek >= preferences.emailFrequencyLimit) {
      // Try in-app only
      return { allowed: true, deliveryMethod: 'in_app_only' };
    }
  }

  return { allowed: true };
}
```

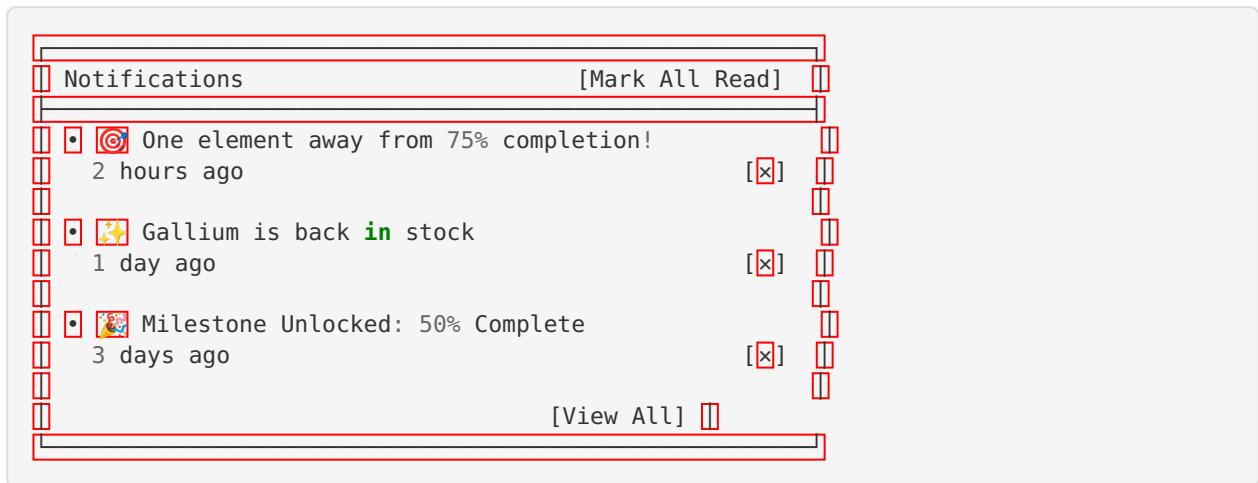
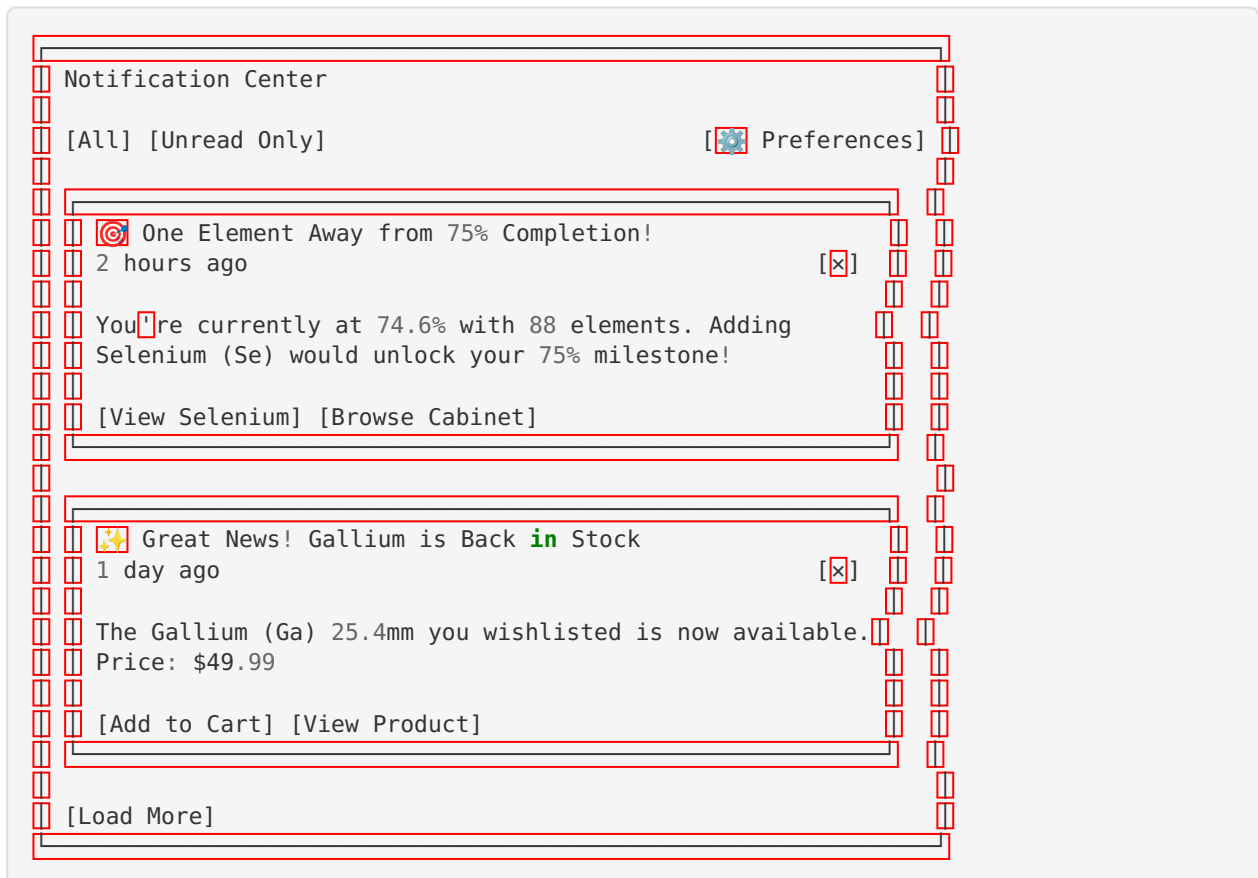
In-App Notification Center

UI Components

Notification Bell Icon (in top nav):



← Shows unread count

Notification Dropdown (click bell):**Full Notification Center Page** (/app/notifications):

Notification Preferences Panel

Notification Preferences [Save]

Email Notifications:

- ☒ Completion Unlock (when 1 element unlocks milestone)
- ☒ Near-Completion (90%+ **in** a category)
- ☒ Wishlist Restock (wishlist items back **in** stock)
- ☒ High-Relevance Arrivals (new products matching interests)
- ☒ Milestone Celebrations (collection milestones)

In-App Notifications:

- ☒ Completion Unlock
- ☒ Near-Completion
- ☒ Wishlist Restock
- ☒ High-Relevance Arrivals
- ☒ Milestone Celebrations

Frequency:

Max emails per week: [3 ▼]

☐ Mute all notifications until: [Date Picker]

[Cancel] [Save]

Email Delivery

Email Service Integration

Recommended: SendGrid (Shopify-compatible, reliable)

Alternative: SMTP via Nodemailer

Email Templates: Use HTML emails with Luciteria branding

Sample Email Structure:


```

<!DOCTYPE html>
<html>
<head>
  <style>
    body { font-family: 'Helvetica Neue', Arial, sans-serif; background-color:
#f4f4f4; }
    .container { max-width: 600px; margin: 0 auto; background: white; padding: 30px; }
    .header { background: linear-gradient(135deg, #667eea 0%, #764ba2 100%); color: wh
ite; padding: 20px; text-align: center; }
    .content { padding: 20px; line-height: 1.6; }
    .cta-button { background: #667eea; color: white; padding: 12px 24px; text-decora-
tion: none; border-radius: 5px; display: inline-block; margin: 10px 0; }
    .footer { text-align: center; padding: 20px; color: #999; font-size: 12px; }
  </style>
</head>
<body>
  <div class="container">
    <div class="header">
      <h1>🕸 Luciteria</h1>
    </div>
    <div class="content">
      <h2>{{subject}}</h2>
      <p>Hi {{user_name}},</p>

      {{message_body}}

      <a href="{{cta_link}}" class="cta-button">{{cta_text}}</a>
    </div>
    <div class="footer">
      <p>You're receiving this because you're a Luciteria collector.</p>
      <p><a href="{{unsubscribe_link}}">Manage Preferences</a></p>
    </div>
  </div>
</body>
</html>

```

Email Sending Logic

```

async function sendEmail(notification) {
  const user = await getUser(notification.userId);
  const preferences = await getUserNotificationPreferences(notification.userId);

  // Check if user wants emails for this type
  const emailEnabled = preferences[`email${notification.type}`];
  if (!emailEnabled) return;

  const template = getEmailTemplate(notification.type);
  const messageData = composeMessage(notification);

  const emailHtml = template
    .replace('{{user_name}}', user.firstName)
    .replace('{{subject}}', messageData.subject)
    .replace('{{message_body}}', messageData.body)
    .replace('{{cta_link}}', messageData.ctaLink)
    .replace('{{cta_text}}', messageData.ctaText)
    .replace('{{unsubscribe_link}}', `${BASE_URL}/app/notifications/preferences`);

  try {
    await sendgrid.send({
      to: user.email,
      from: 'notifications@luciteria.com',
      subject: messageData.subject,
      html: emailHtml
    });

    return { success: true };
  } catch (error) {
    console.error('Email send failed:', error);
    return { success: false, error };
  }
}

```

Testing & Monitoring

Test Scenarios

1. **Completion Unlock**
 - User at 89 elements (1 away from 90% milestone)
 - Add wishlisted in-stock element to cart
 - Verify notification queued
2. **Near-Completion**
 - User owns 6 out of 7 Halogens
 - Add 6th Halogen to collection
 - Verify near-completion notification sent
3. **Wishlist Restock**
 - Product out of stock, on user's wishlist
 - Update inventory to 10 units
 - Verify restock notification sent to all waitlisted users

4. High-Relevance

- Add new product (Rhodium 50mm)
- User has Rh on wishlist
- Verify high-relevance notification sent

5. Milestone

- User adds 59th element
- Verify "50% Dedicated Collector" milestone notification sent

Monitoring Dashboard (for staff)

Metrics to Track:

- Notifications sent per day (by type)
- Email open rates
- Click-through rates on CTAs
- Opt-out rates
- Failed deliveries

Example Dashboard:

Notification Performance (Last 30 Days)

Type	Sent	Opened	Clicked	Opt-Outs
Completion Unlock	247	82.1%	45.3%	2
Near-Completion	189	88.9%	51.2%	1
Wishlist Restock	1,342	76.4%	38.7%	8
High-Relevance	456	69.2%	31.5%	5
Milestone	312	94.6%	67.8%	0

Summary

5 Notification Types:

1. Completion Unlock - Unlocking next milestone
2. Near-Completion - 90%+ category completion
3. Wishlist Restock - Wanted items back in stock
4. High-Relevance Arrival - New products matching interests
5. Milestone Celebration - Collection milestones

Key Features:

- Personalized triggers based on collection state
- Frequency limiting to avoid spam
- User preference controls (per-type opt-in/out)
- Dual delivery (in-app + email)
- Rich, context-aware messaging

Philosophy: Notifications should delight, not annoy. Every alert should be timely, relevant, and actionable.